Season Canon & Scene-First Planning

Three systems that, together, move the story generator from "generate-and-hope" to a pipeline that **plans its forward-references up front**, **guarantees they pay off**, and **gates the result for playability**. The result is seasons that stay internally consistent, never drop a thread, and never strand a player.

The two planning systems are **default-on**: - Season Canon: on, opt-out via config (`seasonCanonEnabled: false`). - Scene-first planning: on, opt-out via `SCENE_FIRST_PLANNING=0` .

The correctness gates run as part of the assembly and final-story contract.

Season Canon

The problem: LLM generation drifts. The same fact — who knows what, whether the bridge is still standing, how a character feels about you — gets re-guessed by every prompt that needs it, and across a multi-episode season those guesses contradict each other and promises get forgotten.

The fix: the moment an episode passes validation, its facts are frozen into a read-only record ("sealing"). Every later prompt **reads** that record instead of reinventing it. It's a five-phase system; the pieces:

The canon store. The frozen, append-only "reality" of a season, holding four kinds of keyed fact: **world facts**, **knowledge** (who-knows-what-when), **character arc state**, and **relationships** (dimensional values by episode). Facts are keyed rather than loose prose so consistency checks are deterministic. Three core operations: - `sealEpisode(N, deltas)` — append episode N's facts. **Append-only by construction**: a fact's establishing episode is fixed, existing facts never mutate, and re-sealing an episode is rejected. - `canonForPrompt(asOfEpisode)` — a read-only snapshot served as "*ESTABLISHED CANON — do not contradict*" into SceneWriter / ChoiceAuthor. - `knownAsOf(characterId, episode)` — the substrate for the **impossible-knowledge gate** (`canonConsistencyValidator`): catches a character acting on something they couldn't yet know.

Persistence (`season-canon.json`) and the LLM fact-extraction step live in the runner, not the store.

Promises: plant → payoff. The **CallbackLedger** tracks "memorable moments": an author tags a notable choice with a `memorableMoment` , the pipeline harvests it into a `CallbackHook` , and unresolved hooks are injected into later prompts until a scene emits a matching payoff. Hardened into a contract: - `payoffEpisode` (P2) names the *specific* episode a promise is due, enforced exactly (the "promise-due gate") rather than against a vague window. - The **SpinePlantMap** (P5) is the deterministic consumer of those targets — it can derive the whole map from the season plan's existing carry-flags (`seasonFlags`), so no LLM schema change is needed, and reports unmatched entries instead of silently dropping them. - `abandoned` / `abandonReason` (P5) explicitly retires a promise whose path was never taken. The completion gate treats abandoned the same as paid.

The completion gate. `validateSeasonCompletion` sweeps the ledger once the season has sealed: every promise must be paid or explicitly retired — never left dangling.

Incremental seal/resume (P4). Episodes seal one at a time with state snapshots, so sealed facts persist across runs and a long season resumes instead of regenerating from scratch.

What you get: no contradictions, no impossible knowledge, no dropped promises, and resumable multi-run seasons.

Scene-First Planning

The old way: the pipeline planned a 7-point story spine, handed each episode one beat, then **invented its scenes on the fly** during generation. Scenes were local and disposable — so a scene could *not* be planned as "the payoff of something two episodes earlier," because no season-wide scene list existed at plan time.

The new way: every episode *and its scenes* are enumerated at the season level first, with the setup→payoff wiring drawn **before any prose is written**. The pieces:

A season-wide scene plan. A `SeasonScenePlan` lists every scene in the season up front, joined by forward-only `SetupPayoffEdge` s. The 7-point structure stays a meta-concept: the season owns the spine, each episode maps to one structural role, and each scene serves its episode's purpose via a `SceneNarrativeRole` (`setup / development / turn / payoff / release`). Beats — the actual prose units — are still generated later, per episode, to serve their pre-planned scene.

Encounters are a kind of scene. A combat or social encounter is just a `PlannedScene` with `kind: 'encounter'` in the same ordered list — so pacing and the consequence budget see it by construction instead of special-casing a parallel encounter structure.

A deterministic v1 builder synthesizes the scene spine from data the season plan already carries (`structuralRole` , `synopsis` , `treatmentGuidance` , consequence chains, etc.), clamped to 3–8 scenes per episode. It works the same for authored treatments and from-scratch stories; an LLM-authored plan can replace it later behind the same flag.

A season-level "dramatic diet." The choice-type mix (expression / relationship / strategic / dilemma ≈ **35/30/20/15**) and the consequence mix (`callback / tint / branchlet / branch` ≈ **50/25/17/8**) are budgeted across the **whole season**, not per episode — so a quiet episode is allowed to be quiet. Scenes weigh 1, encounters weigh 3, and tier invariants hold (an `expression` unit ⇒ `callback` ; a `dilemma` ⇒ at least `branchlet` ; any encounter ⇒ at least `branchlet`). `SeasonBudgetValidator` and `SceneSpineValidator` check this at planning time, **advisory** — the hard gate stays opt-in behind `GATE_SEASON_BUDGETS=1` .

What you get: true long-range setups, first-class encounters, a balanced choice diet authored against rather than trimmed after, and budget problems caught before generation.

Branching & Gameplay Correctness Gates

Planning decides what *should* happen; these gates verify the generated story actually *holds together as something playable*. They run at assembly and final-story time, catching the structural and craft failures that slip past prose-level review — the kind that strand a player or make a choice feel inert.

Routing integrity

The scene graph is the story's wiring; a single bad edge can dead-end a player. The gates check it mechanically:

- **Routing contradictions.** A choice can point one way via `choice.nextSceneId` and another via `leadsTo`; the contract flags `routing_contradiction` and the autofix resolves it by **preferring** `leadsTo` (the authored intent) rather than guessing.
- **Beat-ID collisions.** Beats are numbered per scene (`beat-1`, `beat-2b`…), so the same id — or a hierarchical prefix of it — can recur across scenes. The reader resolves beats per-scene so it isn't a runtime bug, but it's a hazard for saves, analytics, and tooling that resolve ids globally; it's treated as a blocking error and the `StructuralValidator` **namespaces** the colliding scenes at autofix time.
- **Dead-end & empty scenes.** A non-encounter scene with no beats is unplayable (`empty_scene`), and a scene with no forward route strands the player; both are gated rather than shipped.

What you get: no broken links, no soft-locks — every path the planner drew is actually traversable in the reader.

Choice & branch craft

A choice that doesn't visibly change anything is wallpaper. These checks keep choices *meaningful and varied*:

- **Breaking the monoculture.** Stat-checks had collapsed onto a few skills (persuasion alone carried ~43%); post-architecture helpers in `choiceTypePlanner` redistribute checks and break the "dilemma + persuasion" sameness so episodes feel texturally different.
- **Seeded flag callbacks.** Flag-setting choices default to the callback tier, so a decision is wired to be referenced later rather than forgotten.
- **Fork measurement.** The `BranchMechanicalDivergenceValidator` measures real routing forks — does a branch choice actually lead somewhere different — instead of trusting that a "choice" branches.
- **Branch residue in prose.** A branch must leave a trace: the SceneWriter callback is required to carry **branch/path residue** so later prose visibly echoes the path taken (`missing_branch_residue`), making consequences felt, not just tracked.

What you get: choices that diverge mechanically *and* read as if they mattered.

Encounter quality

Encounters are the gameplay spikes; a templated one is the most jarring failure a player can hit. The `EncounterQualityValidator` (blocking, final stage) guards the class structural checks were blind to — bespoke content vs. filler:

- **Template-collapse scan.** Any generic-template fragment (`TEMPLATE_SIGNATURES`) in player-facing encounter prose triggers `encounter_template_collapse` — caught from the final story alone, independent of telemetry.
- **Clock-coverage.** A single-phase encounter whose goal clock outstrips its authored choices is an unfillable gap; the gate flags it once `shrinkClockToCoverage` remediation has had its chance to right-size the clock.
- **Shrink-to-coverage remediation.** Rather than failing outright, the pipeline first shrinks an over-scoped clock down to what was actually authored, reserving the hard block for genuine gaps.

What you get: encounters that always present authored, resolvable content — never boilerplate, never an unwinnable clock.

How they work together

Scene-first planning draws the forward-references — "this choice sets up that payoff." Season Canon registers each as a tracked promise with a concrete `payoffEpisode`, freezes the facts each episode establishes, and refuses to let the season finish with any promise unpaid. **One plans the connections; the other guarantees they hold.**

A worked example: a choice in Episode 1 that pays off in Episode 3

A 3-episode season. In Episode 1 the player can **spare or execute** a captured smuggler, *Renna*. The payoff lands in Episode 3 — spared, she returns as an ally; executed, her crew comes hunting.

1. **Plan time.** The season planner lays out the Episode 1 choice scene *and* the Episode 3 payoff scene up front, joined by a forward edge. The choice is typed `dilemma` and budgeted as a real `branch`. Because it pays off later, it becomes a `SpinePlantEntry` — a **promise due in Episode 3**. `SeasonBudgetValidator` confirms the diet still balances before the plan finalizes.
2. **Episode 1 seals.** The chosen branch sets `renna_spared`. The choice is logged as a `CallbackHook` (pinned to `payoffEpisode: 3`), and the episode's facts are frozen via `sealEpisode`: *Renna spared; Renna knows the player let her live; trust +2.* Append-only — never quietly rewritten.
3. **Episode 2 reads canon.** `canonForPrompt(2)` makes any scene mentioning Renna write her as alive and aware — no re-guessing. The promise isn't due yet, but stays visibly in flight; `knownAsOf` guards against her acting on what she can't know.
4. **Episode 3 pays off.** The pre-planned payoff scene is authored. Because `renna_spared` is read straight from sealed canon, the branch is correct by construction; emitting the matching payoff satisfies the promise-due gate. (Had that path been cut, the hook would be `abandon`-ed instead — never silently dropped.)
5. **Season completes.** `validateSeasonCompletion` confirms every promise was paid or retired. The only failure mode is a thread left dangling — exactly what this architecture prevents.

Stage	Scene-first planning	Season Canon
Plan	lays out Ep1 choice + Ep3 payoff with a forward edge; budgets the dilemma/branch	registers the later payoff as a `SpinePlantEntry`
Ep1	authors the budgeted scene	logs the `CallbackHook`, seals the facts
Ep2	—	serves frozen canon; holds the open promise; guards knowledge
Ep3	authors the pre-planned payoff	records the payoff (or abandons it)
End	—	completion gate: paid or retired, never dangling